
Customer Churn Analysis

An End-to-End Data Science & Machine Learning Project

SQL Server · Python / XGBoost · Power BI · Streamlit · scikit-learn · Jupyter



FINAL PROJECT REPORT

Submitted By

Shehraz Sarwar

Submitted To

Dr. Mirza Adnan Baig

Semester

Pre-Fall 2026

Acknowledgment

All praise is due to Almighty Allah, who granted us the ability, patience, and opportunity to complete this project and course successfully.

I would like to express my sincere gratitude to our instructor, Dr. Mirza Adnan Baig, for his continuous guidance, patience, and constructive feedback throughout the Data Science and Tools & Techniques for Data Science courses. His lectures and practical sessions provided the essential foundation we needed to progress from a raw dataset to a fully functioning, deployable prediction system.

I am also grateful to the Faculty of Engineering Sciences and Technology for fostering an academic environment that encourages practical learning. Finally, I want to thank our classmates for their collaborative discussions and peer reviews, which helped me refine my work and improve the overall quality of this project.

Contents

1	Problem Statement	4
2	Objectives	4
2.1	General Objective	4
2.2	Specific Objectives	4
2.3	Expected Outcomes	4
3	Project Description	4
3.1	Overview	5
3.2	Background	5
3.3	Scope	5
3.4	Working Methodology	5
3.5	Technologies Used	5
3.6	Overall Implementation Approach	5
3.7	Expected Outputs	6
3.8	Real-World Applications	6
4	Methodology	6
4.1	Data Collection	6
4.1.1	Dataset Source	6
4.1.2	Dataset Description	7
4.2	Data Cleaning	7
4.3	Exploratory Data Analysis (EDA)	7
4.4	Feature Engineering	13
4.5	Feature Selection	13
4.6	Data Preprocessing	13
4.6.1	Handling Missing Values	13
4.6.2	Encoding	13
4.6.3	Scaling / Normalization	13
4.6.4	Data Augmentation	14
4.6.5	Train-Test Split	14
4.7	Model Development	14
4.7.1	Model Training	14
4.7.2	Hyperparameter Tuning	14
4.7.3	Model Evaluation & Performance Metrics	15
4.7.4	Model Comparison	15
4.7.5	Deployment	16
5	Project Flowchart	16
6	Mathematical Formula / Calculations	16
7	Python Implementation	17
7.1	Dataset Information	17
7.2	Python Libraries Used	17
7.3	Major Functionalities Implemented	18
8	Output, Charts & Graphs	18
8.1	Power BI Summary Dashboard (Historical Data)	18
8.2	Prediction on the 411 New Joiners	19

8.3	Power BI Prediction Dashboard	19
8.4	ChurnRadar – Streamlit Dashboard	20
9	Key Findings & Business Insights	21
9.1	Top Churn Risk Factors	21
9.2	Retention Strategy Recommendations	21
10	Conclusion	21
11	References	22
12	Author Introduction	23

1 Problem Statement

Customer churn is the point at which a paying customer stops using a company's service. For any subscription-driven business – telecom, banking, SaaS, or streaming – churn is expensive: acquiring a new customer typically costs far more than retaining an existing one, so a business that only reacts *after* a customer has already left is always going to be one step behind.

The real challenge is not just knowing *how many* customers churned last quarter, it is knowing *which* currently active customers are most likely to leave *next*, and why. Most organisations already sit on the raw data needed to answer this (billing history, contract type, service usage, support interactions), but that data is usually scattered across operational databases and never turned into a forward-looking, actionable signal.

This project addresses that gap directly. Using a telecom customer dataset as a working example, we built a complete pipeline that ingests raw customer records, cleans and explores them, trains a machine learning model to recognise the behavioural signature of a churning customer, and finally turns that model into two live tools – a Power BI prediction page and a Streamlit web app called **ChurnRadar** – that a non-technical retention team can open and immediately see which customers to call first.

2 Objectives

2.1 General Objective

To design and deploy an end-to-end customer churn analysis and prediction system that identifies at-risk customers before they leave, and presents that information through interactive dashboards that business users can act on without writing any code.

2.2 Specific Objectives

- Build a reproducible ETL pipeline in SQL Server that ingests raw customer data, audits it for data-quality issues, and produces a clean, analysis-ready production table.
- Perform exploratory data analysis to uncover the demographic, account, and service-related factors that are associated with higher churn.
- Train and compare multiple classification algorithms, then tune the best-performing one so that it balances overall accuracy with a business-acceptable recall floor.
- Deploy the final model as a live prediction tool (ChurnRadar, built with Streamlit) and integrate its output into a Power BI dashboard for executive-level reporting.
- Translate the analysis into concrete, prioritised retention recommendations for the business.

2.3 Expected Outcomes

- A cleaned and validated SQL Server database (**db_Churn**) with staging and production layers.
- A trained and tuned XGBoost classifier, serialised and ready for deployment, along with a documented comparison against four other candidate models.
- A ranked list of newly joined customers flagged as at-risk, complete with churn probability scores.
- Two working interfaces – a Power BI prediction dashboard and a Streamlit application – that surface this list to a retention team in real time.

3 Project Description

3.1 Overview

This project covers every layer of a modern, production-style data pipeline for a single business problem: telecom customer churn. Raw data is ingested and transformed in Microsoft SQL Server, explored and reported on through a two-page Power BI dashboard, modelled in Python using scikit-learn and XGBoost, and finally served through ChurnRadar, a Streamlit application that any retention agent can open in a browser.

3.2 Background

The dataset used here comes from a telecom provider, but the churn signals we explore – contract length, payment behaviour, service bundling, tenure, and geography – have direct equivalents in almost every subscription business. A bank sees the same pattern in account inactivity and fee disputes, a SaaS company sees it in declining feature usage, and a streaming service sees it in days-since-last-login. The architecture built here is intentionally domain-agnostic so that the same pipeline could be repointed at a different dataset with minimal changes.

3.3 Scope

The project covers data ingestion, cleaning, exploratory analysis, feature engineering, model training and tuning, and deployment through two dashboards. It does not cover live streaming data ingestion, automated model re-training (retraining is currently a manual notebook re-run), or integration with an actual CRM/outreach system – the output is a ranked, exportable list that a retention team would still act on manually.

3.4 Working Methodology

The project follows the standard data science lifecycle: data collection → cleaning → exploratory analysis → preprocessing → model training and evaluation → hyperparameter tuning → deployment → prediction on new, unlabeled customers → dashboard reporting. Each of these stages is described in full detail in Section 4 (Methodology).

3.5 Technologies Used

Layer	Tools & Technologies
Data Storage	Microsoft SQL Server – T-SQL, BULK INSERT, Views
Business Intelligence	Power BI Desktop – DAX, slicers, KPI cards, charts
Programming Language	Python 3.12 (Jupyter Notebook + Streamlit)
ML Libraries	scikit-learn, XGBoost, LightGBM, joblib
Data Handling	Pandas, NumPy
Visualisation	Matplotlib, Seaborn
Live Dashboard	Streamlit (ChurnRadar application)
Model Persistence	joblib serialisation (<code>.joblib</code> files)
DB Connector	SQLAlchemy + pyodbc (ODBC Driver 17)

3.6 Overall Implementation Approach

Instead of jumping straight into modelling, we deliberately built the SQL and BI layers first. This meant the null-handling logic, the churned/stayed/joined split, and the KPI definitions were all agreed upon and validated *before* a single line of scikit-learn code was written, which saved us from re-doing preprocessing decisions later.

3.7 Expected Outputs

A cleaned production database, a two-page Power BI report, a tuned XGBoost model with an accompanying preprocessing pipeline, a CSV of flagged at-risk customers, and a live ChurnRadar Streamlit dashboard.

3.8 Real-World Applications

Retention teams can use the ChurnRadar risk list to prioritise outreach calls by probability score. Marketing teams can use the churn-category breakdown (competitor attraction, price, dissatisfaction) to design targeted counter-offers. Regional managers can use the state-level churn map to allocate customer success staff where they are needed most.

4 Methodology

This section walks through every stage of the pipeline in the order it was actually built and run – from raw data to a deployed prediction, not the other way around.

4.1 Data Collection

Customer records were first loaded as a flat CSV and ingested into a SQL Server staging table (`dbo.stg_Churn`) using `BULK INSERT`. The database (`db_Churn`) is dropped and recreated at the start of the script every time, which keeps the pipeline fully reproducible.

```

1  USE master;
2  GO
3  IF EXISTS (SELECT 1 FROM sys.databases WHERE name = 'db_Churn')
4  BEGIN
5      ALTER DATABASE db_Churn SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
6      DROP DATABASE db_Churn;
7  END;
8  GO
9  CREATE DATABASE db_Churn;
10 GO
11 USE db_Churn;
12 GO
13
14 CREATE TABLE dbo.stg_Churn (
15     Customer_ID VARCHAR(50), Gender VARCHAR(20), Age INT,
16     Married VARCHAR(10), State VARCHAR(100),
17     Number_of_Referrals INT, Tenure_in_Months INT,
18     -- ... 24 more columns (services, financials, labels) ...
19     Customer_Status VARCHAR(50), Churn_Category VARCHAR(100),
20     Churn_Reason VARCHAR(255)
21 );
22 GO
23
24 BULK INSERT dbo.stg_Churn
25 FROM 'C:\...\Customer_Data.csv'
26 WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', TABLOCK);
27 GO

```

Listing 1: ETL: database creation and raw data ingestion (abridged)

4.1.1 Dataset Source

The raw dataset is a telecom customer-churn CSV file (32 columns, 6,418 rows) sourced directly from Kaggle (<https://www.kaggle.com/datasets/nguyenduongthanhthuy/telecom-churn-dataset/data>). It is frequently used in SQL and Power BI portfolio projects on the platform, and was ingested locally into our pipeline through SQL Server rather than downloaded fresh from an API.

4.1.2 Dataset Description

The table below summarises the dataset at a glance.

Attribute	Value
Total records	6,418
Total columns	32
Historical (labeled) records	6,007 – Customer_Status $\in$ {Churned, Stayed}
New Joiners (unlabeled)	411 – Customer_Status = Joined (prediction target)
Total historical churners	1,732 (28.8% churn rate)
Target variable	Customer_Status (binary: Churned = 1, Stayed = 0)

Feature groups: **Demographics** (Gender, Age, Married, State, Number_of_Referrals), **Account Info** (Tenure_in_Months, Value_Deal, Contract, Paperless_Billing, Payment_Method), **Services** (Phone_Service, Multiple_Lines, Internet_Type, Online_Security, Online_Backup, Device_Protection_Plan, Premium_Support, Streaming_TV/Movies/Music, Unlimited_Data), and **Financials** (Monthly_Charge, Total_Charges, Total_Refunds, Total_Extra_Data_Charges, Total_Long_Distance_Charges, Total_Revenue).

4.2 Data Cleaning

A single null-audit query (using the `COUNT(*) - COUNT(column)` pattern across all 32 columns) gave us a complete null-profile of the dataset in one pass. The imputation logic was domain-aware rather than mechanical: a null in a service column almost always means the customer simply does not subscribe to that service, so it is filled with 'No' or 'None' rather than dropped. Churn classification nulls (for customers who never churned) are grouped into 'Others'. This logic was applied inline with `ISNULL()` while writing the cleaned data into the production table `dbo.prod_Churn`, and two views (`vw_ChurnData` for historical records, `vw_JoinData` for new joiners) were created on top of it to keep the ML training data and the prediction targets cleanly separated.

At the Python level, we also confirmed there were zero fully-duplicated rows and zero duplicated `Customer_ID` values in the historical dataset, so no further de-duplication was needed.

4.3 Exploratory Data Analysis (EDA)

EDA was performed entirely in Python on the 6,007 historical records pulled from `vw_ChurnData`, before any encoding or model training.

Outlier check. A boxplot grid across all nine numerical columns (Figure 1) showed genuine but extreme values in `Total_Refunds`, `Total_Extra_Data_Charges`, and `Total_Long_Distance_Charges`. We chose not to remove them – these are real high-value customers with unusual billing profiles, and dropping them would hurt the model's ability to recognise exactly the customers the business most wants to keep.

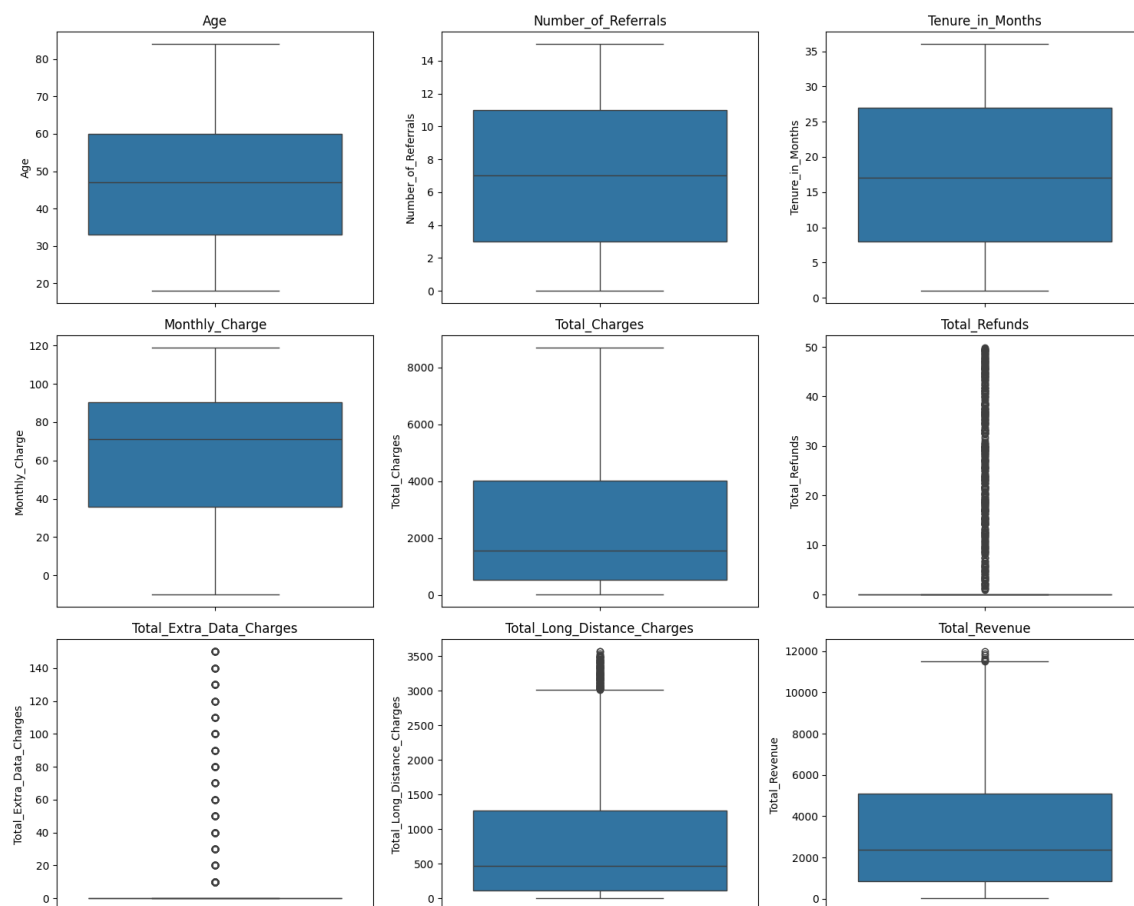


Figure 1: Boxplot grid across all nine numerical features – outlier spread in Age, Tenure, Charges, and Revenue columns.

Correlation check. A Pearson correlation heatmap (Figure 2) was computed against the binary-encoded target. **Total_Charges** and **Total_Revenue** are almost perfectly correlated (0.97), which is expected since revenue is derived from charges. Against the churn target itself, correlations stay weak-to-moderate: **Total_Revenue** (-0.28) and **Total_Long_Distance_Charges** (-0.27) are the strongest, followed by **Total_Charges** (-0.25), while **Monthly_Charge** sits lower at 0.16 – confirming that no single numerical feature dominates the churn signal on its own, which is exactly why a tree-based ensemble model was chosen over a simple linear classifier.

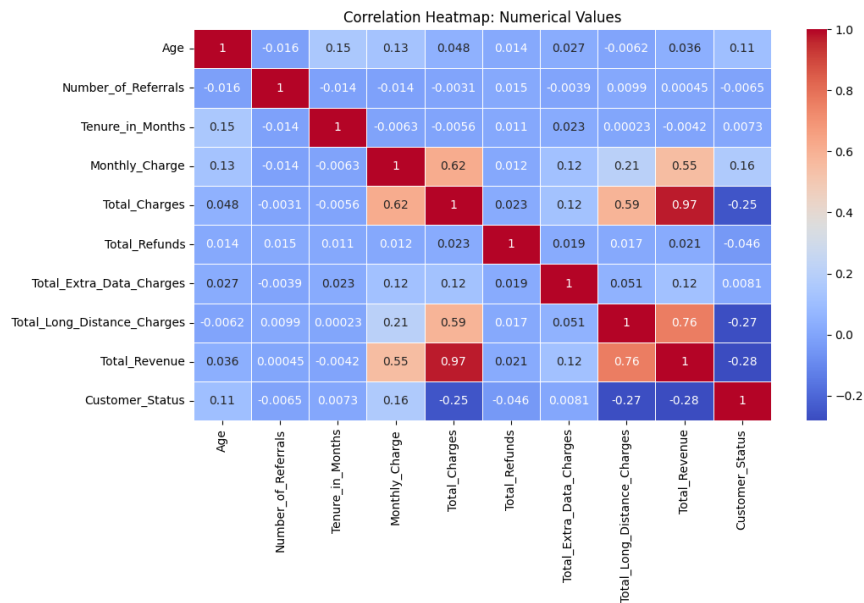


Figure 2: Pearson correlation heatmap of numerical features against Customer_Status.

Demographic and behavioural patterns. Age and tenure were bucketed into groups to make the churn-rate breakdowns easier to read business-wise.

```

1 df_churn['Age_Group'] = pd.cut(
2     df_churn['Age'], bins=[0, 20, 35, 50, 200],
3     labels=['<20', '20-35', '35-50', '>50']
4 )
5 df_churn['Tenure_Group'] = pd.cut(
6     df_churn['Tenure_in_Months'], bins=[-1, 6, 12, 18, 24, 1000],
7     labels=['<6 Months', '6-12 Months', '12-18 Months',
8           '18-24 Months', '>=24 Months']
9 )

```

Listing 2: Age and tenure bucketing used throughout the EDA

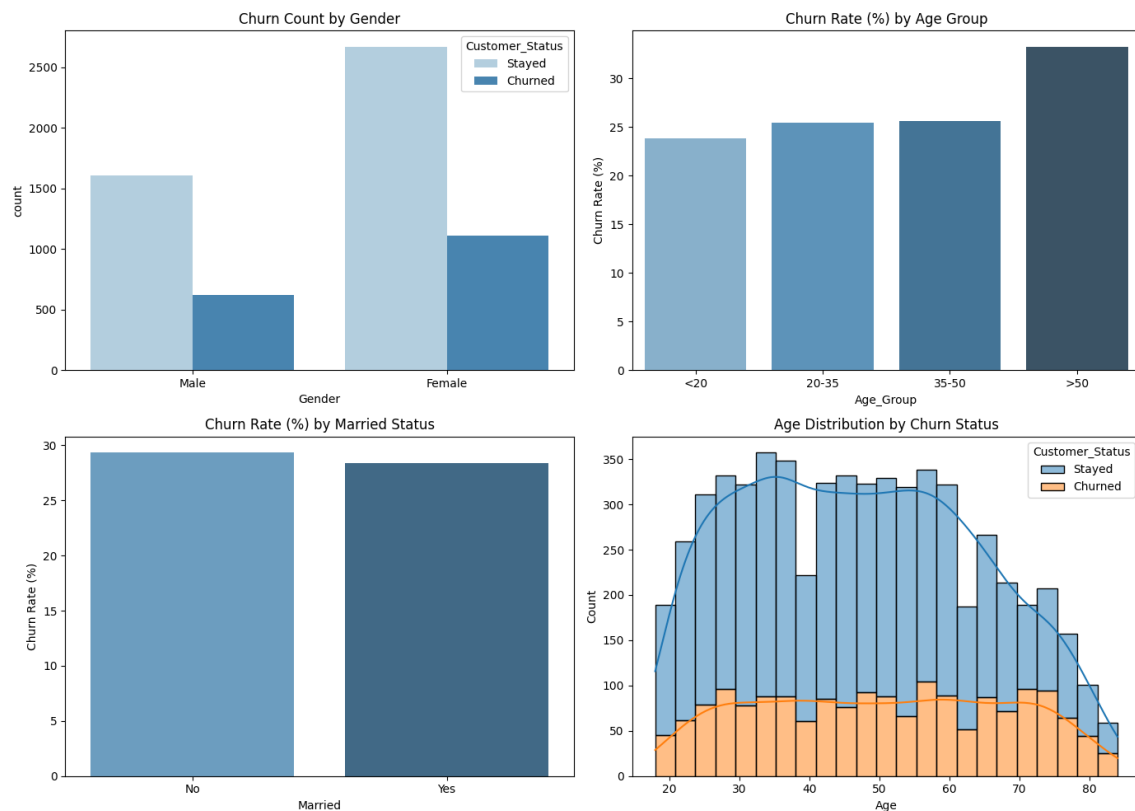


Figure 3: Churn count by gender, churn rate by age group, churn rate by marital status, and age distribution by churn status.

Female customers churn noticeably more in absolute count than male customers, and churn rate climbs steadily with age – customers above 50 have the highest churn rate of any age group.

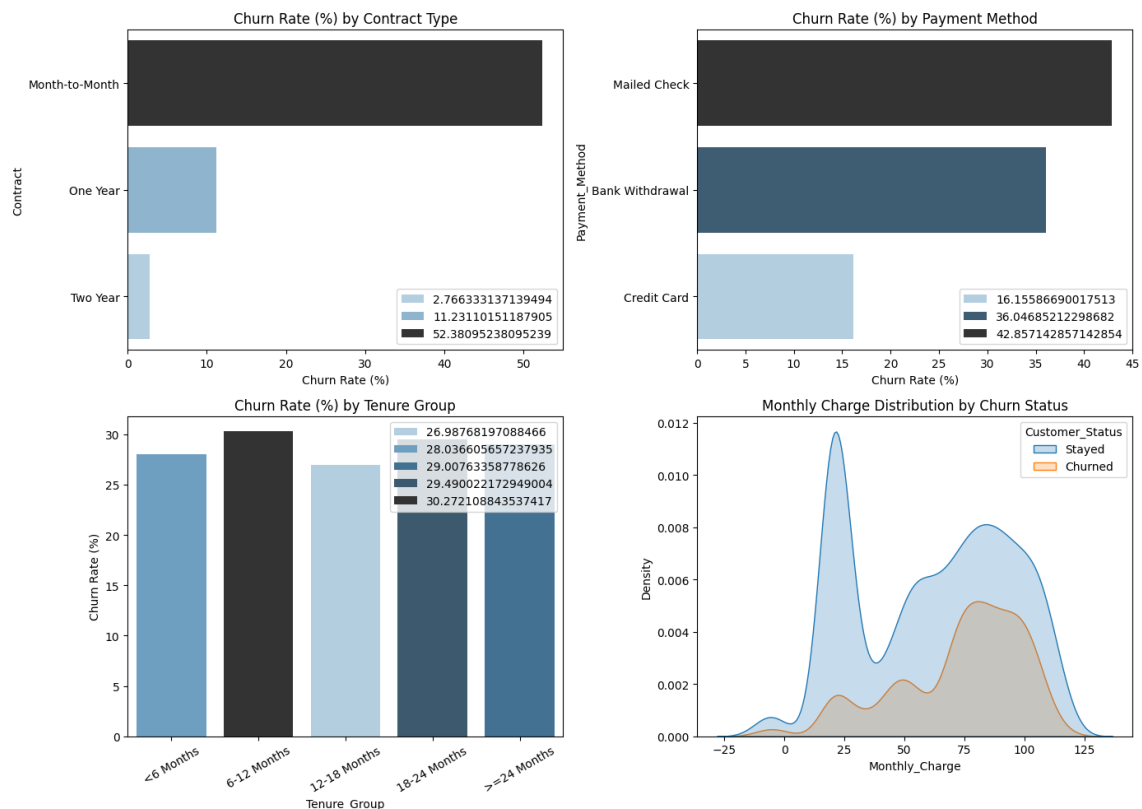


Figure 4: Churn rate by contract type, payment method, tenure group, and monthly charge distribution.

Month-to-Month customers churn at roughly 52.4%, against 11.2% for One Year and 2.8% for Two Year contracts – by far the strongest structural signal in the whole dataset. Payment method shows a similar gradient: Mailed Check customers churn at 42.9%, Bank Withdrawal at 36.0%, and Credit Card at just 16.2%, suggesting that digital-first payment habits track with general account engagement.

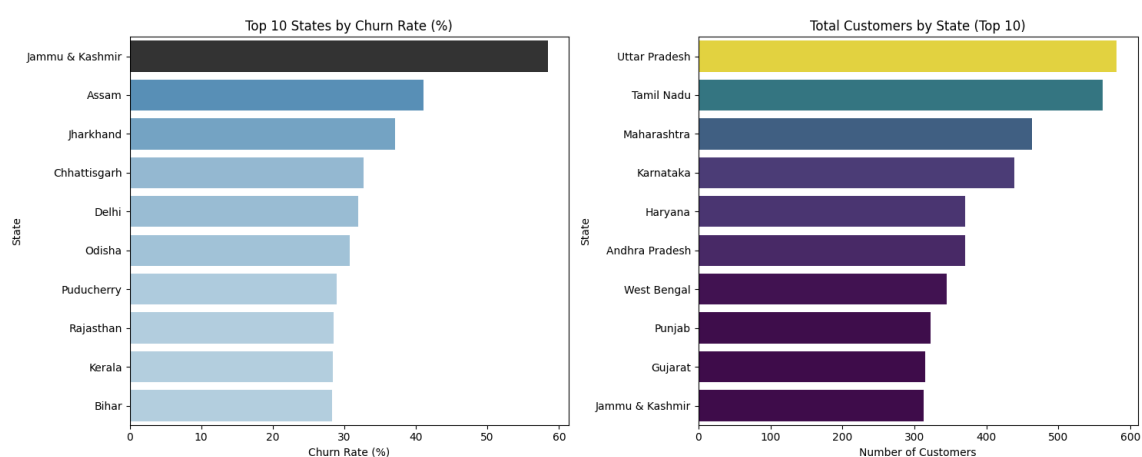


Figure 5: Top 10 states by churn rate, and total customers by state (top 10 by volume).

Jammu & Kashmir stands out sharply with a churn rate near 58%, followed by Assam and Jharkhand. Interestingly, the states with the largest customer bases (Uttar Pradesh, Tamil Nadu, Maharashtra) are not the same states with the worst churn rates – which matters for

how the business should prioritise regional retention spend.

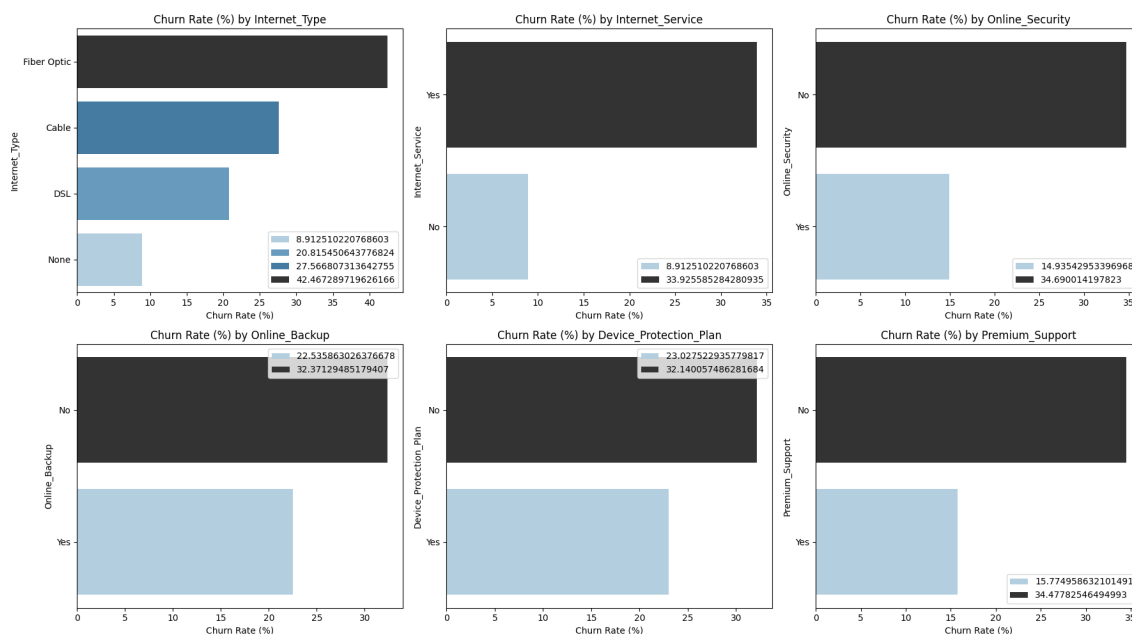


Figure 6: Churn rate by internet type and by add-on service subscription (security, backup, device protection, premium support).

Fiber Optic customers churn at 42.5%, well above Cable (27.6%) and DSL (20.8%), pointing to a service-quality or pricing issue specific to that tier. Across every add-on service we checked, customers *without* the service consistently churn more than customers *with* it – the gap is largest for Premium Support (34.5% vs 15.8%) and Online Security (34.7% vs 14.9%), a strong practical argument for bundling.

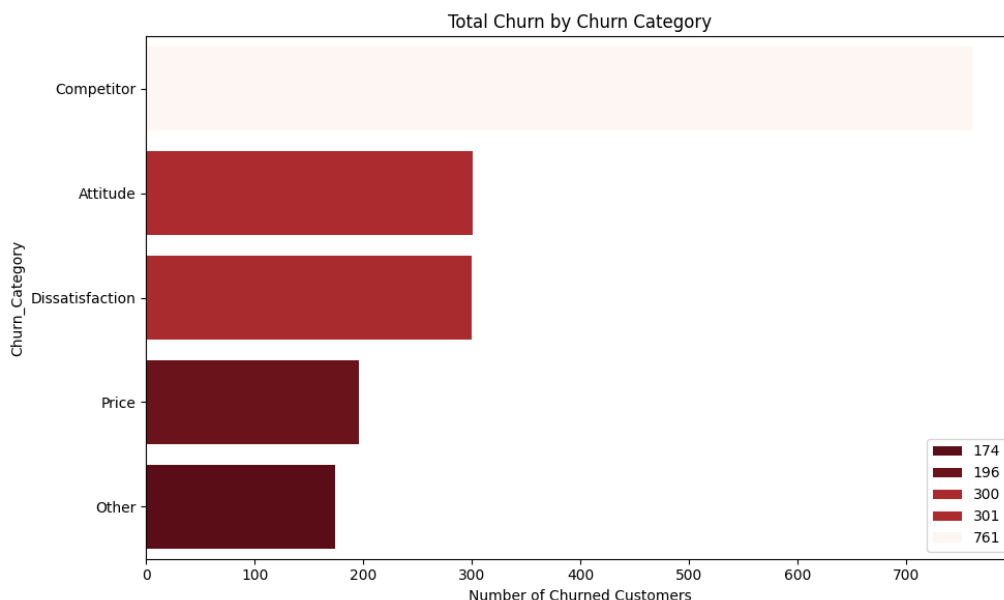


Figure 7: Total churned customers by stated churn category.

Competitor attraction is by far the single largest reason customers give for leaving (761 of 1,732 churners, roughly 44%), ahead of attitude issues, general dissatisfaction, and price.

4.4 Feature Engineering

`Age_Group` and `Tenure_Group` were engineered purely for readable EDA charts and were *not* passed into the model – the raw `Age` and `Tenure_in_Months` columns were kept for training so the model could learn finer-grained thresholds than our manual buckets.

4.5 Feature Selection

No formal feature-selection algorithm (e.g. RFE or SelectKBest) was used. Instead, columns that would leak the target – `Churn_Category`, `Churn_Reason`, `Customer_ID`, and `Customer_Status` itself – were dropped before training, and every remaining feature was kept. Tree-based ensembles handle irrelevant or weakly-correlated features gracefully, so aggressive manual pruning was judged unnecessary.

4.6 Data Preprocessing

4.6.1 Handling Missing Values

All missing-value handling was already completed in the SQL layer (Section 4.2), so the Python pipeline receives a fully populated `DataFrame`.

4.6.2 Encoding

Categorical columns were split into nominal (no natural order) and ordinal (natural order) groups and encoded accordingly through a single `ColumnTransformer`.

```

1  from sklearn.compose import ColumnTransformer
2  from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
3
4  nominal_cols = [
5      "Gender", "Married", "State", "Value_Deal", "Phone_Service",
6      "Multiple_Lines", "Internet_Service", "Internet_Type",
7      "Online_Security", "Online_Backup", "Device_Protection_Plan",
8      "Premium_Support", "Streaming_TV", "Streaming_Movies",
9      "Streaming_Music", "Unlimited_Data", "Paperless_Billing",
10     "Payment_Method",
11 ]
12 ordinal_cols = ["Contract"]
13 contract_order = [["Month-to-Month", "One Year", "Two Year"]]
14
15 preprocessor = ColumnTransformer(
16     transformers=[
17         ("ord", OrdinalEncoder(categories=contract_order), ordinal_cols),
18         ("nom", OneHotEncoder(handle_unknown="ignore",
19                               sparse_output=False), nominal_cols),
20     ],
21     remainder="passthrough", # numerical columns pass through unchanged
22 )
23 preprocessor.set_output(transform="pandas")
24
25 X_train_processed = preprocessor.fit_transform(X_train)
26 X_test_processed  = preprocessor.transform(X_test)

```

Listing 3: Preprocessing pipeline: ordinal + one-hot encoding

4.6.3 Scaling / Normalization

No feature scaling was applied. Every candidate model in this project (Decision Tree, Naive Bayes, Random Forest, XGBoost, LightGBM) is either tree-based or, in the case of Naive Bayes, insensitive to feature magnitude in the way linear/distance-based models are, so scaling would not have changed model behaviour.

4.6.4 Data Augmentation

Not applicable – this is a structured tabular dataset, and class imbalance (roughly 29% churned vs 71% stayed) was instead handled through class-weighting parameters (`class_weight="balanced"`, `scale_pos_weight`, `is_unbalance=True`) rather than synthetic oversampling.

4.6.5 Train-Test Split

An 80/20 stratified split was used so both sets preserve the same churn/stayed ratio.

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.2, random_state=42, stratify=y
3 )
4 print(f"Training set shape: {X_train.shape}") # (4805, 28)
5 print(f"Testing set shape: {X_test.shape}")   # (1202, 28)

```

Listing 4: Stratified 80/20 train-test split

4.7 Model Development

4.7.1 Model Training

Five classifiers were trained on the identical preprocessed training set: Decision Tree, Gaussian Naive Bayes, Random Forest, XGBoost, and LightGBM. Ensemble models used 150 estimators with built-in class-imbalance handling.

```

1 models = {
2     "Decision Tree": DecisionTreeClassifier(random_state=42, class_weight="balanced"),
3     "Naive Bayes": GaussianNB(),
4     "Random Forest": RandomForestClassifier(
5         n_estimators=150, max_depth=10, random_state=42,
6         class_weight="balanced"),
7     "XGBoost": XGBClassifier(
8         n_estimators=150, learning_rate=0.08, max_depth=5,
9         random_state=42, scale_pos_weight=2, eval_metric="logloss"),
10    "LightGBM": LGBMClassifier(
11        n_estimators=150, learning_rate=0.08, max_depth=5,
12        random_state=42, is_unbalance=True),
13 }
14
15 for name, model in models.items():
16     model.fit(X_train_processed, y_train)
17     y_pred = model.predict(X_test_processed)
18     print(name, accuracy_score(y_test, y_pred),
19           precision_score(y_test, y_pred),
20           recall_score(y_test, y_pred), f1_score(y_test, y_pred))

```

Listing 5: Training and evaluating five candidate classifiers

4.7.2 Hyperparameter Tuning

The baseline XGBoost model was then tuned using `RandomizedSearchCV` (100 candidate combinations, 5-fold stratified cross-validation) over a wide parameter grid. Rather than optimising raw accuracy alone, a custom scorer was used that rewards accuracy *only if* recall stays above a business-acceptable floor of 71% – otherwise the combination is heavily penalised. This directly encodes the business reality that a missed churner (false negative) is far more costly than a false alarm.

```

1 def accuracy_with_recall_floor(y_true, y_pred, min_recall=0.71):
2     rec = recall_score(y_true, y_pred)
3     if rec < min_recall:
4         return rec - 1.0 # heavily penalise combos below the floor
5     return accuracy_score(y_true, y_pred)
6
7 custom_scorer = make_scorer(accuracy_with_recall_floor)
8

```

```

9 random_search = RandomizedSearchCV(
10     estimator=XGBClassifier(random_state=42, eval_metric='logloss'),
11     param_distributions=param_grid, # n_estimators, max_depth, lr, ...
12     n_iter=100, scoring=custom_scorer,
13     cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
14     random_state=42, n_jobs=-1,
15 )
16 random_search.fit(X_train_processed, y_train)
17 tuned_model = random_search.best_estimator_

```

Listing 6: Custom recall-floor scorer used for hyperparameter search

The search selected: `n_estimators=200`, `max_depth=7`, `learning_rate=0.05`, `subsample=0.6`, `colsample_bytree=0.7`, `min_child_weight=5`, `gamma=0`, `reg_alpha=1`, `reg_lambda=2`, `scale_pos_weight=2`.

4.7.3 Model Evaluation & Performance Metrics

Standard classification metrics were used throughout: Accuracy, Precision, Recall, and F1-Score (formal definitions in Section 5). All five baseline models were evaluated on the same held-out 1,202-record test set.

4.7.4 Model Comparison

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	0.7887	0.6084	0.7522	0.6727
Naive Bayes	0.7745	0.5837	0.7637	0.6617
Random Forest	0.8378	0.7123	0.7349	0.7234
XGBoost (baseline)	0.8386	0.7131	0.7378	0.7252
LightGBM	0.8228	0.6642	0.7810	0.7179
XGBoost (tuned)	0.8461	0.7355	0.7291	0.7323

The tuned XGBoost model improves accuracy, precision, and F1-score over every other candidate while keeping recall comfortably above the 71% floor we set going in. It was therefore selected as the final, deployed model – a change from our earlier baseline-only deployment, made possible once we widened the tuning search and enforced the recall constraint explicitly rather than trusting the search to preserve recall on its own.

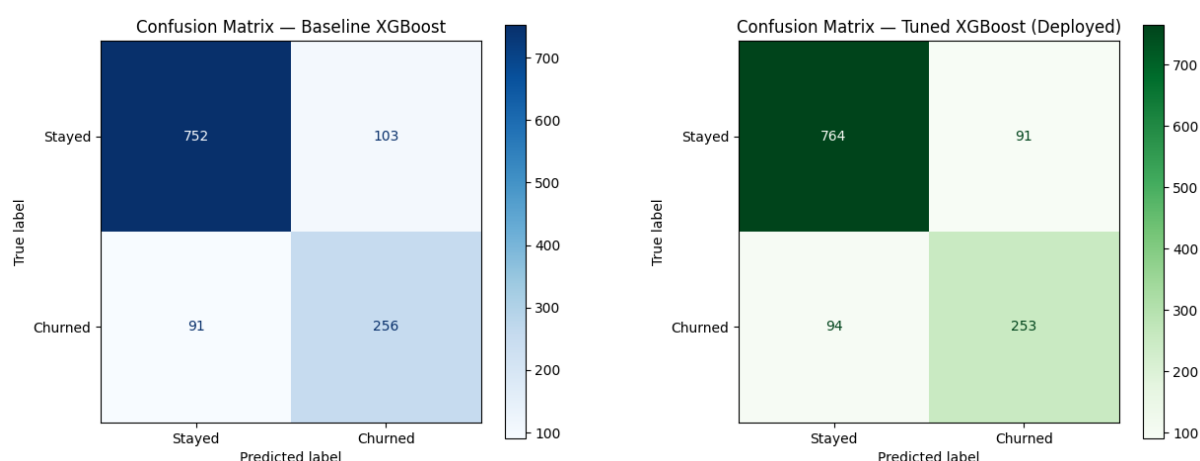


Figure 8: Confusion matrix: baseline XGBoost (left) vs. tuned XGBoost – the deployed model (right). Tuning reduced false positives from 103 to 91 while keeping false negatives nearly unchanged (91 → 94).

4.7.5 Deployment

The fitted preprocessing pipeline and the tuned XGBoost model were serialised with `joblib` and shipped as two files: `churn_preprocessor.joblib` and `tuned_xgb_model.joblib`. Both are loaded at runtime by the Power BI prediction workflow and by the ChurnRadar Streamlit app, so retraining the model never requires touching the dashboard code.

```
1 joblib.dump(preprocessor, "churn_preprocessor.joblib")
2 joblib.dump(baseline_model, "baseline_xgb_model.joblib") # kept for reference
3 joblib.dump(final_model, "tuned_xgb_model.joblib")      # deployed model
```

Listing 7: Serialising the production pipeline

5 Project Flowchart

Figure 9 shows the full path data takes through this project, from the raw CSV file to the two live dashboards a retention team actually uses.

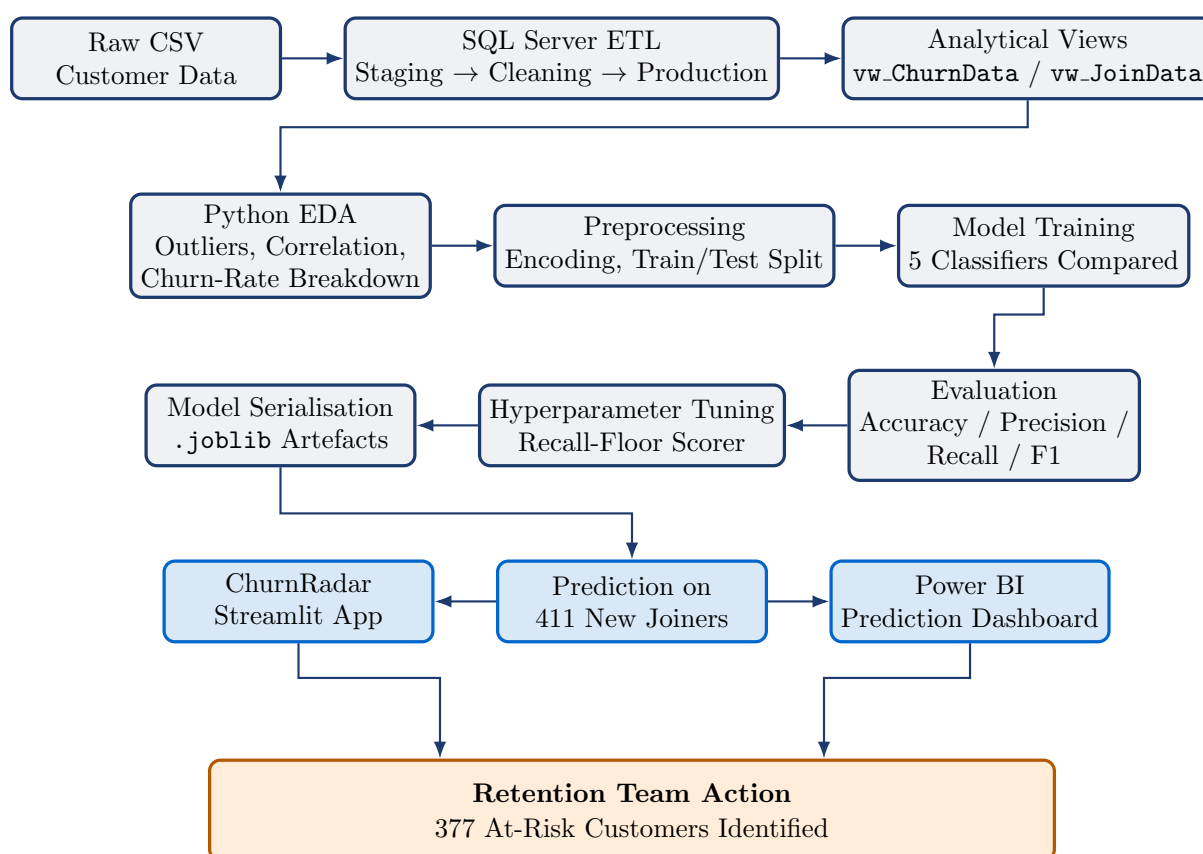


Figure 9: End-to-end project workflow: from raw data ingestion through model development, prediction, and retention action.

6 Mathematical Formula / Calculations

This section lists the formulas used to build features, train the model, and score its predictions.

Churn probability (model output). XGBoost produces a raw score z for each customer, converted to a probability with the logistic (sigmoid) function:

$$p(\text{Churn} = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

A customer is flagged as at-risk when $p \geq 0.50$ (the default ChurnRadar threshold), and marked *Critical* when $p \geq 0.90$.

XGBoost training objective. At each boosting round t , the model minimises:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

where l is the logistic loss, T is the number of leaves in tree f_t , and γ, λ are the regularisation terms tuned in Section 4.7.2.

Evaluation metrics, computed from the confusion matrix (TP, TN, FP, FN):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Custom tuning scorer (recall-floor rule used inside `RandomizedSearchCV`):

$$\text{score}(y, \hat{y}) = \begin{cases} \text{Recall}(y, \hat{y}) - 1, & \text{Recall}(y, \hat{y}) < 0.71 \\ \text{Accuracy}(y, \hat{y}), & \text{Recall}(y, \hat{y}) \geq 0.71 \end{cases}$$

Pearson correlation, used for the EDA heatmap in Figure 2:

$$r_{xy} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$$

Churn rate for a category c (used across every EDA bar chart):

$$\text{ChurnRate}(c) = \frac{\text{Number of churned customers in } c}{\text{Total customers in } c} \times 100\%$$

7 Python Implementation

7.1 Dataset Information

Dataset Name	Telecom Customer Churn Dataset (<code>Customer_Data.csv</code>)
Dataset Source	Course-provided telecom churn dataset, loaded via SQL Server
Number of Records	6,418 (6,007 historical + 411 new joiners)
Number of Columns	32
Features	Demographic, account, service, and financial attributes
Target Variable	<code>Customer_Status</code> (Churned = 1, Stayed = 0)

7.2 Python Libraries Used

- **pandas, numpy** – data loading, cleaning, and manipulation
- **matplotlib, seaborn** – exploratory visualisation
- **sqlalchemy, pyodbc** – connecting to SQL Server from the notebook
- **scikit-learn** – preprocessing, train-test split, Decision Tree, Naive Bayes, Random Forest, `RandomizedSearchCV`, evaluation metrics
- **xgboost, lightgbm** – gradient-boosted classifiers
- **joblib** – model and pipeline serialisation
- **streamlit** – the ChurnRadar web application

7.3 Major Functionalities Implemented

- **Database Connectivity** – SQLAlchemy/pyodbc integration pulling the vw_ChurnData and vw_JoinData views from SQL Server into Pandas DataFrames
- **Classification** – binary churn prediction (Churned vs Stayed)
- **Ensemble Learning** – Random Forest, XGBoost, and LightGBM
- **Hyperparameter Optimisation** – randomised search with a custom, business-aware scoring function
- **Batch Prediction & Export** – scoring the 411 new joiners with the deployed model and exporting the flagged at-risk list to CSV

```

1 engine = create_engine(
2     "mssql+pyodbc://localhost\\SQLEXPRESS/db_Churn"
3     "?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes"
4 )
5 conn = engine.connect()
6
7 df_churn = pd.read_sql("SELECT * FROM vw_ChurnData", conn) # 6,007 rows, labeled
8 df_join  = pd.read_sql("SELECT * FROM vw_JoinData", conn)  # 411 rows, unlabeled

```

Listing 8: Connecting to SQL Server and extracting the two working DataFrames

```

1 loaded_preprocessor = joblib.load("churn_preprocessor.joblib")
2 loaded_model = joblib.load("tuned_xgb_model.joblib")
3
4 X_new = df_join.drop(columns=["Customer_ID", "Churn_Category",
5                             "Churn_Reason", "Customer_Status"],
6                      errors="ignore")
7 X_new_processed = loaded_preprocessor.transform(X_new)
8
9 df_join["Churn_Prediction"] = loaded_model.predict(X_new_processed)
10 df_join["Churn_Probability"] = loaded_model.predict_proba(X_new_processed)[: , 1]
11
12 high_risk_customers = (
13     df_join[df_join["Churn_Prediction"] == 1]
14     .sort_values("Churn_Probability", ascending=False)
15 )
16 print(f"Found {len(high_risk_customers)} customers at risk of churning.")
17 # Output: Found 377 customers at risk of churning.
18
19 high_risk_customers.to_csv("high_risk_churn_list.csv", index=False)

```

Listing 9: Scoring the 411 new joiners with the deployed model

8 Output, Charts & Graphs

8.1 Power BI Summary Dashboard (Historical Data)

The first Power BI page reports on the 6,007 historical customers with slicers for Monthly Charge Range and Marital Status.

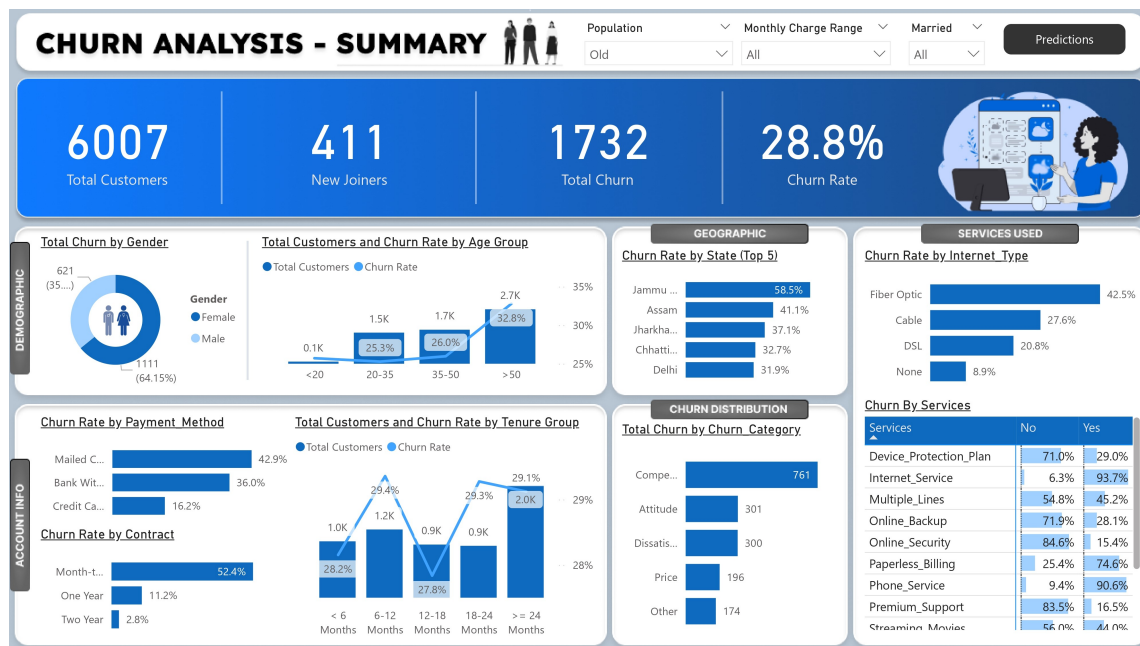


Figure 10: Power BI Summary Dashboard – KPIs, demographic analysis, service usage, geographic churn, and churn category breakdown.

8.2 Prediction on the 411 New Joiners

Once the tuned XGBoost model was applied to the 411 new joiners at the default 50% threshold, **377 customers** were flagged as at risk, carrying a combined **Rs. 43,772** in revenue.

Predicted Churner Profile	Breakdown
Gender	Female: 246 Male: 131
Age Group	<20: 12 20-35: 105 35-50: 128 >50: 132
Marital Status	No: 198 Yes: 179
Contract	Month-to-Month: 362 One Year: 15 Two Year: 0
Tenure Group	<6m: 65 6-12m: 90 12-18m: 57 18-24m: 61 ≥24m: 104
Payment Method	Credit Card: 192 Bank Withdrawal: 148 Mailed Check: 37
Internet Type	None: 149 DSL: 98 Fiber Optic: 79 Cable: 51
Top States	Uttar Pradesh (43), Maharashtra (39), Tamil Nadu (36), Karnataka (30)

8.3 Power BI Prediction Dashboard

The second Power BI page integrates the model's exported predictions directly, so stakeholders can view and act on churn risk without touching any Python code.

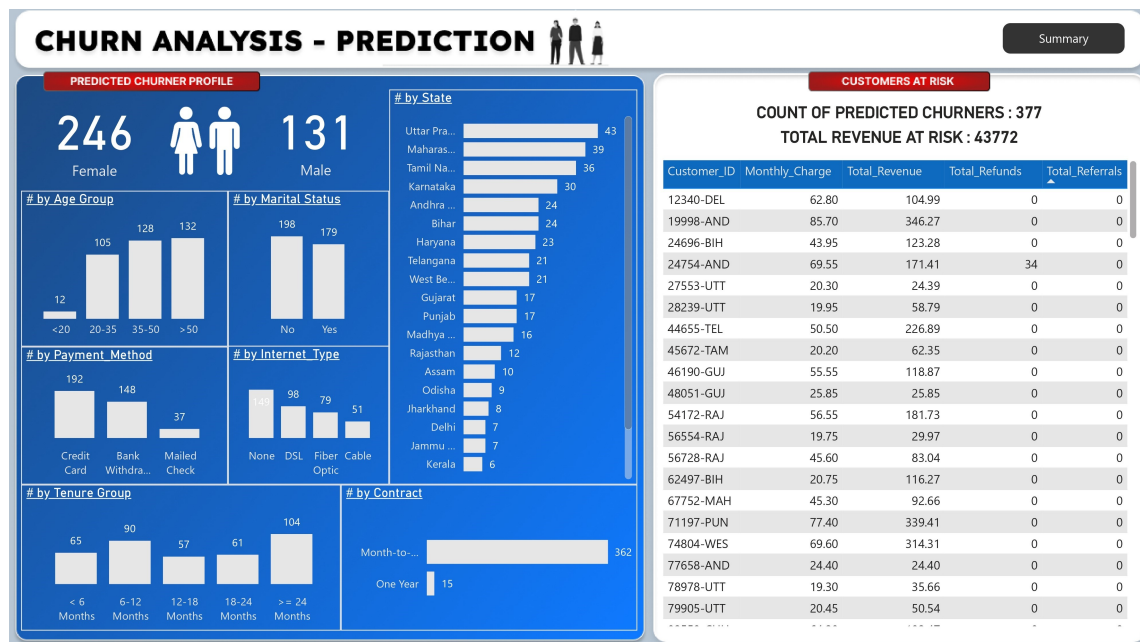


Figure 11: Power BI Prediction Dashboard – tuned XGBoost predicted-churner profile (377 customers, Rs. 43,772 revenue at risk) and the at-risk customer detail table.

8.4 ChurnRadar – Streamlit Dashboard

ChurnRadar lets a non-technical user drag and drop the two `.joblib` files and a customer CSV, and immediately see the at-risk list – no notebook, no command line beyond launching the app.

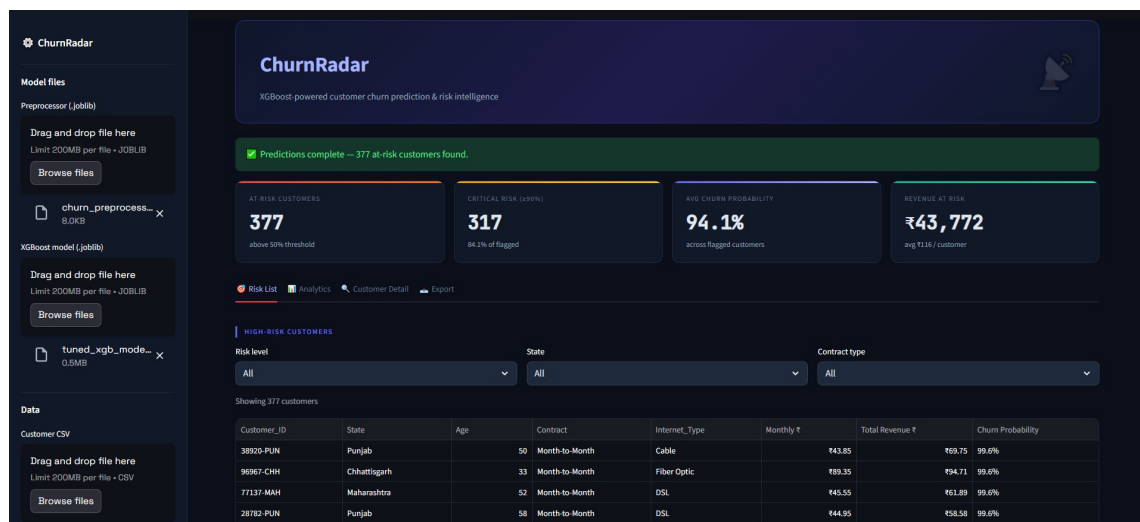


Figure 12: ChurnRadar Streamlit dashboard – 377 at-risk customers found, 317 of them (84.1%) in the Critical risk band, average churn probability 94.1%, Rs. 43,772 total revenue at risk.

Dashboard tabs: Risk List (filterable by risk level, state, and contract type), Analytics (churn probability histogram, top states, Monthly Charge vs. Churn Probability scatter), Customer Detail (individual profile lookup), and Export (one-click CSV download of the filtered list).

Risk classification:

Badge	Probability Range	Action Priority
Critical	$\geq 90\%$	Immediate outreach (317 customers, 84.1% of flagged)
High	75%–89%	Priority queue
Medium	50%–74%	Monitor and engage

9 Key Findings & Business Insights

9.1 Top Churn Risk Factors

1. **Contract Type (Month-to-Month)** – roughly 52% historical churn rate, and 362 of the 377 newly-flagged at-risk customers are on this contract type. It remains the single most powerful predictor in the dataset.
2. **Competitor Offers** – 761 historical churners explicitly cited a competitor, the largest single churn category by far.
3. **Fiber Optic Internet** – 42.5% historical churn rate, clearly above Cable and DSL.
4. **Mailed Check Payment** – 42.9% churn rate, the highest of the three payment methods.
5. **Age Above 50** – the highest churn rate of any age band, and the largest single group (132 of 377) in the newly-flagged list.
6. **Geographic Hotspots** – Jammu & Kashmir, Assam, and Jharkhand lead historical churn; Uttar Pradesh, Maharashtra, and Tamil Nadu lead the forward-looking at-risk list by volume.
7. **Low Service Bundling** – customers without Premium Support or Online Security churn at more than double the rate of those with it.

9.2 Retention Strategy Recommendations

- **Contract upgrade incentives** for Month-to-Month customers, especially the 362 already flagged as at risk.
- **A rapid-response counter-offer script** for customers showing intent to switch to a competitor.
- **A service-quality review of the Fiber Optic tier**, given its consistently high churn rate.
- **Bundle-and-save promotions** for customers with no add-on security or support subscription.
- **Immediate outreach to the 317 Critical-risk customers** identified by ChurnRadar, before any further model refresh.

10 Conclusion

- This project delivered a working, end-to-end churn pipeline that goes from a raw CSV file to a live, browser-based prediction tool – not just a notebook that ends at a metric.
- All stated objectives were achieved: a cleaned SQL Server database, a two-page Power BI report, five compared classifiers, a tuned and deployed XGBoost model, and two live dashboards.
- The final tuned XGBoost model reached 84.6% accuracy, 73.6% precision, and 72.9% recall on the held-out test set, and was chosen specifically because it improved on the baseline without violating a business-defined recall floor.
- Applying it to the 411 new joiners flagged 377 customers, worth Rs. 43,772 in revenue, as at-risk – with 317 of them in the Critical band requiring immediate attention.
- The main challenge was balancing statistical performance against business cost: a model that is technically more “accurate” is not automatically the right one to deploy if it quietly

lets more churners slip through, which is why the custom recall-floor scorer became the central design decision in this project.

- Future improvements could include automated periodic retraining, integrating outreach outcomes back into the training data, and adding a time-series component to flag *when*, not just *whether*, a customer is likely to leave.

11 References

1. Kaggle Dataset: Telecom Customer Churn Dataset. <https://www.kaggle.com/datasets/nguyenduongthanhthuy/telecom-churn-dataset/data>
2. CampusX. (Director). *Decision Trees Geometric Intuition — Entropy — Gini impurity — Information Gain* [Video]. YouTube. <https://youtu.be/IZnno-dKgVQ>
3. CampusX. (Director). *Introduction to XGBOOST — Machine Learning* [Video]. YouTube. <https://youtu.be/C6aDw4y8qJ0>
4. Unfold Data Science. (Director). *GridSearchCV vs RandomizedSearchCV* [Video]. YouTube. <https://www.youtube.com/watch?v=w4frwjt8uCo>
5. Hussain, S., & Shamim, S. S. *AI Lecture on Decision Trees* [Lecture notes and presentation]. Faculty of Engineering Sciences and Technology, Iqra University.
6. Sarwar, S. *Handwritten Notes on Basic to Advanced Python Data Analysis and SQL Server* [Personal academic notes]. Iqra University.

12 Author Introduction

Assalam-u-Alaikum,

I am **Shehraz Sarwar**, a dedicated student currently pursuing my undergraduate studies in Computer Science at Iqra University. I maintain a strong academic record with a CGPA of **3.94** and continuously strive to expand my technical knowledge while applying innovative technologies to solve real-world challenges.

My primary areas of interest include **Artificial Intelligence, Machine Learning, Data Science, and Software Development**. Throughout my academic journey, I have worked on multiple practical projects that have strengthened my technical expertise and enhanced my problem-solving abilities. These experiences have enabled me to bridge the gap between theoretical concepts and real-world applications.

In addition to my academic projects, I have gained valuable hands-on experience through internships and collaborative development work. As a Data Science Intern at **10Pearls Pakistan**, I built end-to-end MLOps pipelines, automated model retraining using GitHub Actions, and deployed real-time AQI forecasting systems. Furthermore, during my selection in the competitive Summer Internship Program at the **State Bank of Pakistan**, I engineered a 7-stage Python pipeline leveraging web scraping, PDF parsing, and Gemini AI to automate the extraction and standardization of corporate financial statements. These professional experiences have significantly strengthened my technical, analytical, and communication skills while working in enterprise environments.

As a lifelong learner, I remain committed to exploring emerging technologies, enhancing my skill set, and contributing to impactful innovations in the fields of Artificial Intelligence and Software Engineering.



END OF REPORT
